

程序设计的实验部分

Bring color to knowledge

Author: 胡潇天 (后简称活性炭)

Institute: 假装有工作室

Date: 2025 年 12 月 26 日

Version: v6.0.0

邮箱: 1985716106@qq.com



人生是一场漫长的旅程，不在乎目的地，而在乎沿途的风景以及看风景的心情。让我们带着希望和勇气，去探索未知的世界，去追求更高的目标



第 1 章 lab1 素数分析器

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

1.1 介绍

在本实验中，你将创建一个素数分布器，一个素数分布器可以有多重多样的功能，比如判断一个数是否是素数，寻找 1 到 n 中的所有素数，输出素数之和等功能

1.1.1 项目

下载 lab1.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab1.cpp

1.1.2 学习内容

lab1 的学习内容较 lab0 较为庞大，知识量也较为多，请注意不要投机取巧采用一些简单的方式进行运算，而是强迫自己使用文档想要教给你的东西

- 素数的判断以及优化
- 如何使用动态数组 vector 的一些基本知识
- 如何使用迭代器以及基于范围的 for 循环
- 如何合理的使用头文件中的各种强大函数

1.1.3 完成内容

逐步 is_prime(),memory_prime,iteration(),sum() 等多个函数

1.2 Getting Started

1.2.1 Phase 1

执行 is_prime, 这个函数能够帮助我们去判断一个数是否是素数, 他需要一个参数:

- x: 传入一个任意的自然数

is_prime 函数，若一个数为素数则返回 true，若一个数是合数或者既不是质数也不是合数则返回 false

Axiom 1.1 (Hint)

- 对于一个数 n 来说，我们需要从 2 开始判断是否能够整除该数，这个数如果能够整除 n，则存在因子，不是素数。
- 从某种意义上将，我们只需要遍历到 $\sqrt{n}$ 即可，不需要遍历到 n，可以想想为什么。

1.2.2 Phase 2

执行 memory_prime 函数，能够将 1 到 n 中所有的素数存储在一个叫 primes 的数组中，但是根据常规的方法，你可以想到，假设我这个 n 是非常大的，int 以内的素数大约有一亿个，我需要开一个这么大的数组，但如果我的 n 很小，又会导致我开的一亿大小的数组被浪费掉了很多空间，所以我们这里引入动态数组的概念。

Definition 1.1 (vector)

vector 是 *C++* 标准模板库（*STL*）中的动态数组。和普通数组不同，*vector* 的长度可以动态增长或缩小。内存是连续分配的，所以可以像数组一样通过下标访问元素。

你可能需要以下知识:

```
vector<int> primes; //初始化一个动态数组,叫primes
int num ; cin >> num ;
primes.push_back(num); //向primes中插入一个num.
```

该函数有两个参数:

- vector<int>primes, 传入一个你需要修改的动态数组，向其中插入所有的素数
- n 你需要插入的范围的上确界。

1.2.3 Phase 3

1.2.3.1 基于范围的 for 循环

执行 iteration 函数，但是你发现有两个这样类型的函数，但是一个后面使用了 for 循环，这里我们的实验要求使用一种特殊的 for 循环，称为基于范围的 for 循环，用于遍历其中的元素。可以理解成从一个数组中取出一个值，然后对其操作。

你可能需要以下的代码形式:

```
for(const int & x : v)
{
    cout << x ;
}
```

使用冒号来分割，可以理解成 v 中的元素取出来到了 x 里面，然后完成输出的操作之后，自动的进行下一个数的操作，继续输出，但是这里注意的是 const 和 & 符号，其中 const 确保了里面的值不会被修改，而 & 是需要修改 v 中值的时候用的。~~这不左右脑互相博弈吗?~~ 这里的 & 起到的作用是防止在正常遍历过程中产生大量的副本，其实就是值传递的本质，而引用传递会确保不会产生副本加快程序的运行速度。

1.2.3.2 使用迭代器进行迭代

对于第二个版本的迭代你需要一个叫迭代器的功能来实现，说白了就是指向第一个数的指针

Definition 1.2 (迭代器)

迭代器（*iterator*）是 *STL*（标准模板库）中的一个“泛化指针”。它的作用是用统一的方式访问容器里的元素，而不用关心容器底层是数组、链表、树还是哈希表。可以把迭代器理解为指向容器中元素的对象。

请参考一下代码进行迭代器的循环使用

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4};
    // 定义迭代器
    vector<int>::iterator it;
    // 让迭代器指向容器开头
    it = v.begin();
    cout << *it << endl; // 输出 1
    it++;
    cout << *it << endl; // 输出2

    it = v.end();
    cout << *it << endl; // 输出4
}
```

ps: 最终输出格式为 1 2 3 4 5 这样，中间用空格隔开然后占用一行。

1.2.4 Phase 4

实现 sum 函数，我想这个时候就有很多人要说了，sum 函数遍历数组即可实现，这里我们采用比较现代化的实现，c++ 库中有非常多好用的多函数，我们的实验部分不需要大家去思考很多高级的算法，只需要去知道有这个东西，毕竟实际开发中是不需要写很多已有的东西的。

- 首先请在头部写好这个函数需要的头文件 #include <numeric>

请使用函数 accumulate(first,end,init) 来实现求和的功能，该函数有三个参数第一个参数是指向数组头部的迭代器，第二个是指向尾部的迭代器，第三个是求和的初始值。

1.3 submit

1.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，只需要提交 cpp 后缀的文件即可，其他的文件不需要提交。

1.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

1.4 References

- 有关 vector 的一些使用
- 基于范围的 for 循环使用